INTRODUCTION TO PROGRAMMING • DIGITAL NOTES

UNIT II

Control Structures

Sequential • if / if-else / switch • for / while / do-while • break & continue

▶ What you will learn in this unit

- Simple sequential programs
- if, if-else, else-if ladder, nested if
- switch-case statement
- Choosing between if and switch
- for loop
- while loop
- do-while loop
- Nested loops & patterns
- break and continue
- Classic programs (prime, factorial...)
- Common mistakes
- Exam questions

2.1 Simple Sequential Programs

◆ Definition

Control structures decide the **order** in which statements run. There are three kinds: **Sequence** (one after another), **Selection** (choose a path: if, switch) and **Iteration** (repeat: loops).

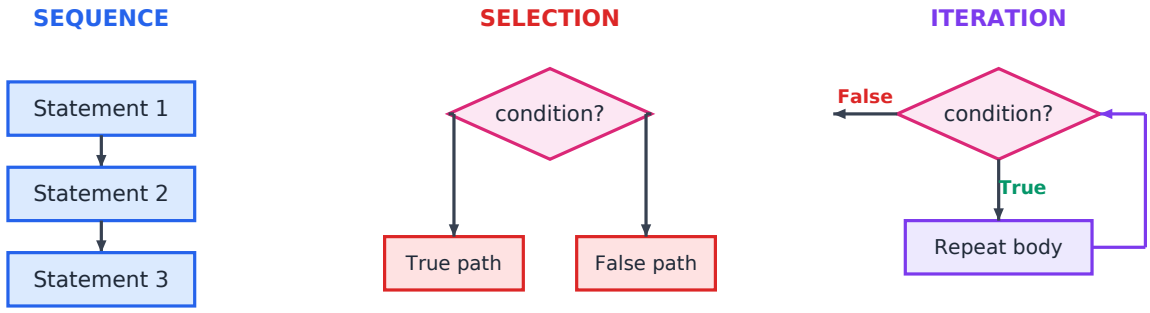


Fig 2.1 — The three basic control structures

A **sequential program** has no decisions or loops — statements run top to bottom. Pattern: **Input → Process → Output**.

Area & perimeter of a rectangle

```c
#include <stdio.h>
int main() {
    float l, b;
    printf("Enter length and breadth: ");
    scanf("%f %f", &l, &b);                    // INPUT
    printf("Area = %.2f\n", l * b);             // PROCESS + OUTPUT
    printf("Perimeter = %.2f\n", 2 * (l + b));
    return 0;
}
```

OUTPUT

```
Enter length and
breadth: 4 5
Area = 20.00
Perimeter = 18.00
```

How to write any program: IPO + dry run

- **Step 1 – Input:** what must the user give? (declare variables)
- **Step 2 – Process:** which formula / decision / repetition turns input into result?
- **Step 3 – Output:** what must be displayed and in what format?
- **Step 4 – Dry run:** test the program on paper with sample values (a **trace table**) before running it.

Celsius to Fahrenheit

```c
float c, f;
printf("Enter temperature in Celsius: ");
scanf("%f", &c);
f = (c * 9 / 5) + 32;                 // formula from the problem
printf("Fahrenheit = %.1f\n", f);
```

OUTPUT

```
Enter temperature in
Celsius: 100
Fahrenheit = 212.0
```

2.2 Conditional Statements

Conditions use relational (`< > == !=`) and logical (`&& || !`) operators. In C, **0 is false** and **any non-zero value is true**.

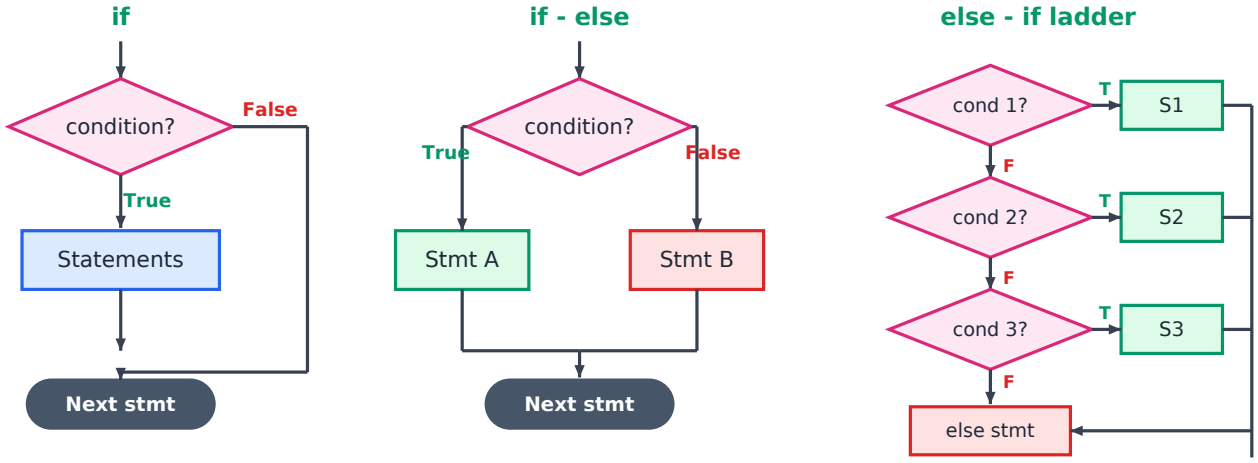


Fig 2.2 — Flowcharts of if, if-else and else-if ladder

Syntax of if family

```c
if (marks >= 40)                  // simple if
    printf("Pass");

if (age >= 18) printf("Adult");   // if-else
else           printf("Minor");

if      (m >= 90) printf("A");    // else-if ladder: first TRUE wins
else if (m >= 75) printf("B");
else if (m >= 60) printf("C");
else              printf("Fail");
```

Nested if — largest of three

```c
int a = 12, b = 45, c = 30, max;
if (a >= b) {                          // NESTED if: an if inside an if
    if (a >= c) max = a; else max = c;
} else {
    if (b >= c) max = b; else max = c;
}
printf("Largest = %d", max);
// Leap year:  (y%4==0 && y%100!=0) || (y%400==0)
```

OUTPUT

```
Largest = 45
```

> ⚠ **Common Mistake**
> `if (a = 5)` **assigns** 5 (always true). Use `==` to compare. Also never put a semicolon after the condition: `if (x > 0);` makes the body run always.

A	B	A && B (AND)	A || B (OR)	!A (NOT)
1 (true)	1 (true)	1	1	0
1 (true)	0 (false)	0	1	0
0 (false)	1 (true)	0	1	1
0 (false)	0 (false)	0	0	1

Conditional (ternary) operator

```
max = (a > b) ? a : b;     // ternary operator = compact if-else:  condition ? if_true : if_false
printf("%s", (n % 2 == 0) ? "Even" : "Odd");
```

switch statement

Use `switch` when one variable/expression is compared with many **constant** values (int or char).

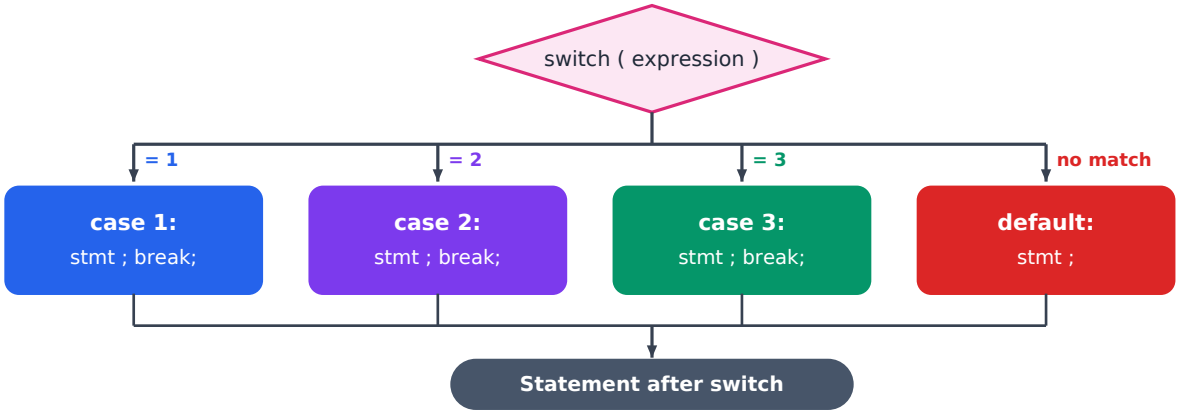


Fig 2.3 — Flow of the switch statement (break jumps out)

Calculator using switch

```
char op; float x, y;
scanf("%f %c %f", &x, &op, &y);
switch (op) {
    case '+': printf("%.2f", x + y); break;
    case '-': printf("%.2f", x - y); break;
    case '*': printf("%.2f", x * y); break;
    case '/': if (y != 0) printf("%.2f", x / y);
              else printf("Divide by zero");
              break;
    default : printf("Invalid operator");   // optional
}
```

OUTPUT

```
Input:  8 * 2.5
Output: 20.00
```

Grouping cases (vowel check)

```
switch (ch) {                  // no break = fall-through
    case 'a': case 'e': case 'i': case 'o': case 'u':
        printf("Vowel");  break;
    default:
        printf("Consonant");
}
```

OUTPUT

```
ch = 'e'  ->  Vowel
```

	if – else	switch
Tests	Ranges and complex conditions (`x > 5 && y < 3`)	Only equality with constants
Data types	Any (int, float, ...)	int and char only (not float / string)
Speed / clarity	Slower for many choices	Cleaner and faster for menus
Default branch	`else`	`default` (optional)

★ Exam Focus
Rules: case labels must be **unique constants**; `break` is needed to stop fall-through; `default` may appear anywhere; switch cannot test ranges or floating-point values.

2.3 Loops (Iteration)

◆ Definition
A **loop** repeats a block of statements while a condition stays true. Every loop has: **1) initialization, 2) condition, 3) update, 4) body**.

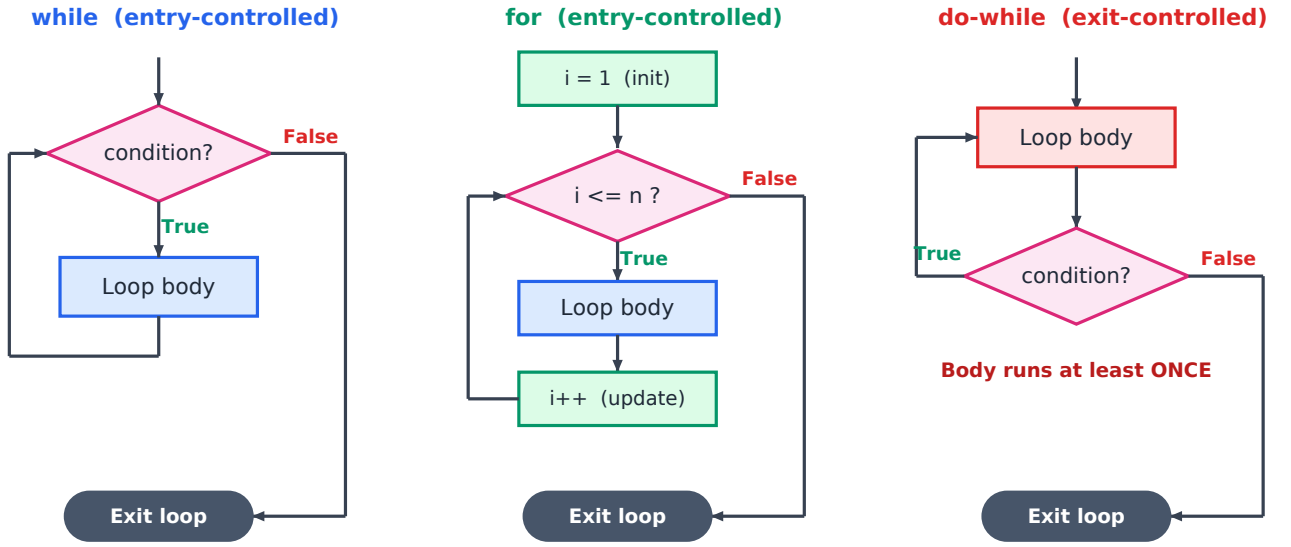


Fig 2.4 — Flowcharts of the three loops

Syntax of loops

```
for (init; condition; update)       // best when the number of repetitions is KNOWN
    body;

init;                               // while: repetitions UNKNOWN, test first
while (condition) { body; update; }

init;                               // do-while: body first, test later (menus, validation)
do { body; update; } while (condition);     // <- note the semicolon!
```

for (init ; condition ; update) { body } — order of execution: 1 → 2 → 4 → 3 → 2 → 4 → 3 … → 2 (false) → exit

1	2	3	4
INITIALIZE `int i = 1`	**CONDITION** `i <= 5`	**UPDATE** `i++`	**BODY** `{ sum += i; }`
once, at start	checked every time	after each pass	runs if true

Fig 2.5 — Anatomy of a for loop

i	Condition i<=5	sum before	sum after (sum += i)
1	true	0	1
2	true	1	3
3	true	3	6
4	true	6	10
5	true	10	15
6	**false → exit**	15	15 (final)

Sum of first 5 numbers (for)

```c
int i, sum = 0;
for (i = 1; i <= 5; i++)       // trace table above
    sum += i;
printf("Sum = %d", sum);
```

OUTPUT

```
Sum = 15
```

while loop

```c
int n = 5, i = 1;
while (i <= 5) {                        // multiplication table of 5
    printf("%d x %d = %d\n", n, i, n * i);
    i++;                                // forgetting this = infinite loop!
}
```

OUTPUT

```
5 x 1 = 5
5 x 2 = 10
...
5 x 5 = 25
```

do-while — input validation

```c
int n;
do {
    printf("Enter n > 0: ");
    scanf("%d", &n);
} while (n <= 0);              // repeat until input is valid
```

OUTPUT

```
Enter n > 0: -4
Enter n > 0: 7
```

Feature	for	while	do-while
Type	Entry-controlled	Entry-controlled	Exit-controlled
Condition checked	Before each pass	Before each pass	**After** each pass
Minimum runs	0	0	**1**
Best used when	Count is known	Count is unknown	Must run at least once (menu)
Semicolon at end	No	No	**Yes**

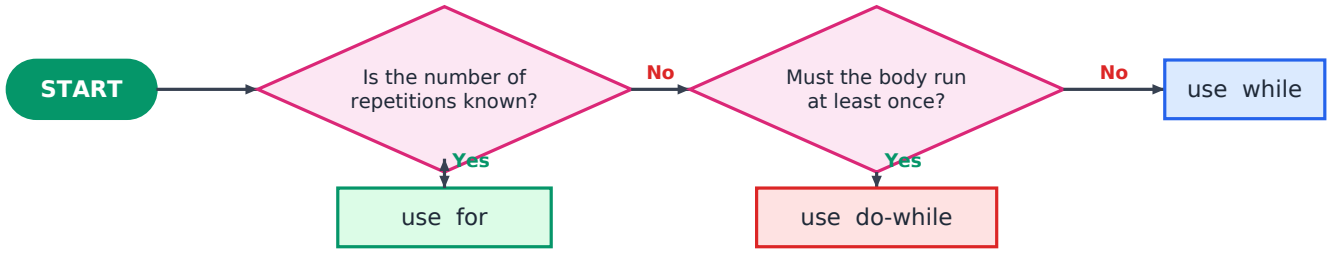


Fig 2.6 — Which loop should I use?

Infinite loops

A loop that never ends: `while(1)`, `for(;;)`, or a condition that never becomes false (missing update). Stop it with `break` or **Ctrl + C**.

Nested loops and patterns

A loop inside another loop: the **inner loop runs fully for every single pass of the outer loop**. Outer loop = rows, inner loop = columns. Total runs = outer × inner.

Star triangle

```c
for (i = 1; i <= 4; i++) {          // rows
    for (j = 1; j <= i; j++)        // columns: i stars in row i
        printf("* ");
    printf("\n");                   // new line after each row
}
```

OUTPUT

```
*
* *
* * *
* * * *
```

Outer i	Inner j runs	Printed in the row	Inner body count
1	j = 1	*	1
2	j = 1, 2	* *	2
3	j = 1, 2, 3	* * *	3
4	j = 1, 2, 3, 4	* * * *	4 → total = 1+2+3+4 = **10**

Pyramid

```c
for (i = 1; i <= 3; i++) {
    for (s = 1; s <= 3 - i; s++) printf(" ");        // leading spaces
    for (j = 1; j <= 2 * i - 1; j++) printf("*");    // stars
    printf("\n");
}
```

OUTPUT

```
  *
 ***
*****
```

More patterns

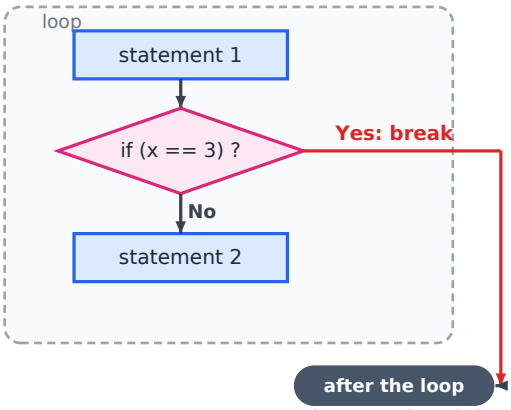
```c
for (i = 1; i <= 4; i++) {            // number triangle
    for (j = 1; j <= i; j++) printf("%d ", j);
    printf("\n");
}
for (i = 3; i >= 1; i--) {            // inverted star triangle
    for (j = 1; j <= i; j++) printf("* ");
    printf("\n");
}
```

OUTPUT

```
1
1 2
1 2 3
1 2 3 4
* * *
* *
*
```

2.4 break and continue

break — exit the loop completely



continue — skip rest, start next pass

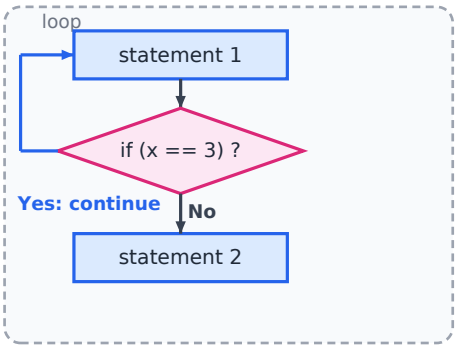


Fig 2.7 — break leaves the loop; continue jumps to the next iteration

break

```c
for (i = 1; i <= 10; i++) {
    if (i == 6) break;          // stop the whole loop at 6
    printf("%d ", i);
}
```

OUTPUT

```
1 2 3 4 5
```

continue

```c
for (i = 1; i <= 5; i++) {
    if (i == 3) continue;       // skip only i = 3
    printf("%d ", i);
}
```

OUTPUT

```
1 2 4 5
```

	break	continue
Effect	Terminates the nearest loop / switch	Skips the rest of the current pass
Control goes to	First statement after the loop	Loop update / condition check
Used in	Loops and **switch**	Loops only
Typical use	Search found, invalid input, prime test	Skip unwanted values

2.5 Classic Programs Using Control Structures

Factorial

```c
long fact = 1; int n = 5, i;
for (i = 1; i <= n; i++) fact *= i;          // 5! = 1x2x3x4x5
printf("%d! = %ld", n, fact);
```

OUTPUT

```
5! = 120
```

Fibonacci series (8 terms)

```c
int a = 0, b = 1, c, i;
for (i = 1; i <= 8; i++) {
    printf("%d ", a);
    c = a + b;  a = b;  b = c;          // add previous two
}
```

OUTPUT

```
0 1 1 2 3 5 8 13
```

Digit sum and reverse

```c
int n = 1234, rev = 0, sum = 0, digit;
while (n > 0) {
    digit = n % 10;            // last digit
    sum  += digit;
    rev   = rev * 10 + digit;  // build reversed number
    n     = n / 10;            // drop last digit
}
printf("Sum=%d Reverse=%d", sum, rev);   // palindrome if n == rev
```

OUTPUT

```
Sum=10 Reverse=4321
```

Prime number check

```c
int n = 29, i, prime = 1;
if (n < 2) prime = 0;
for (i = 2; i * i <= n; i++)            // test divisors up to sqrt(n)
    if (n % i == 0) { prime = 0; break; }   // divisor found -> not prime
printf(prime ? "Prime" : "Not prime");
```

OUTPUT

```
Prime
```

GCD of two numbers

```c
int a = 48, b = 18, t;
while (b != 0) {              // Euclid: replace (a,b) by (b, a mod b)
    t = b;  b = a % b;  a = t;
}
printf("GCD = %d", a);
```

OUTPUT

```
GCD = 6
```

Armstrong number (3 digits)

```c
int n = 153, t = n, dg, s = 0;
while (t > 0) { dg = t % 10;  s += dg * dg * dg;  t /= 10; }
printf(s == n ? "Armstrong" : "Not Armstrong");   // 1+125+27 = 153
```

OUTPUT

```
Armstrong
```

Menu-driven program (do-while + switch)

```c
int ch;
do {
    printf("1.Add 2.Sub 3.Exit : ");
    scanf("%d", &ch);
    switch (ch) {
        case 1: printf("Add selected\n"); break;
        case 2: printf("Sub selected\n"); break;
        case 3: printf("Bye!\n");         break;
        default: printf("Invalid\n");
    }
} while (ch != 3);
```

OUTPUT

```
1.Add 2.Sub 3.Exit : 1
Add selected
1.Add 2.Sub 3.Exit : 3
Bye!
```

● Real-life Use

Loops power menus, ATMs (repeat until Exit), games (game loop), searching/sorting data, billing totals and password retry limits. Selection statements power every login check, grade calculator and traffic-light controller.

Common mistakes at a glance

Mistake	Result	Fix
`if (x = 5)`	Assigns, always true	Use `==`
`for (...);` or `if (...);`	Empty body, next line runs once	Remove the extra **;**
Missing `break` in switch	Fall-through to next cases	Add `break;`
Missing update (`i++`)	Infinite loop	Always change the loop variable
Off-by-one: `<` vs `<=`	One extra / missing pass	Trace first and last value

Predict the output (practice)

#	Code	Output	Why?
1	`for (i=0; i<3; i++) printf("%d ", i*i);`	0 1 4	i takes 0,1,2 and prints squares
2	`int i=10; do { printf("%d ", i); i--; } while (i>8);`	10 9	body first; stops when i = 8
3	`switch(2){ case 1: printf("One"); case 2: printf("Two"); case 3: printf("Three"); break; }`	TwoThree	no break after case 2 → falls through
4	`for (i=1; i<=5; i++) { if (i%2==0) continue; printf("%d", i); }`	135	even values skipped
5	`int x = 0; if (x = 5) printf("Yes");`	Yes	= assigns 5 (non-zero = true)
6	`printf("%d", (5 > 3) && (2 > 4));`	0	true AND false = false (0)
7	`int i=1; while (i<=3) printf("%d ", i);`	infinite 1 1 1 ...	i is never incremented

★ UNIT SUMMARY — QUICK REVISION

- **Sequence, selection, iteration** are the three control structures. In C, **0 = false**, non-zero = true.
- Selection: **if, if-else, else-if ladder, nested if** for conditions; **switch** for many constant choices (needs `break`).
- **for** (count known), **while** (test first), **do-while** (runs at least once; ends with **;**).
- Nested loop: inner loop completes for each outer pass → used for patterns, matrices, tables.
- **break** exits the loop/switch; **continue** skips to the next iteration.
- Digit tricks: `n % 10` = last digit, `n / 10` = remove last digit. Prime test needs divisors only up to √n.

Q Exam-Oriented Questions

Short Answer (2 marks)

1. Name the three basic control structures with one example each.
2. Differentiate between `=` and `==`.
3. What is an else-if ladder? When is it used?
4. What happens if `break` is missing in a switch case?
5. List two differences between while and do-while.
6. What is an infinite loop? Give a code example.
7. What is a nested loop? How many times does the inner statement run for outer=3, inner=4?
8. Differentiate between break and continue.
9. Can a switch use a float variable? Why?
10. Write the syntax of the for loop and explain its parts.

Long Answer / Programs (5–10 marks)

1. Explain all forms of the if statement with flowcharts and examples.
2. Explain the switch statement with syntax, flowchart and a menu-driven calculator program.
3. Compare for, while and do-while loops with flowcharts and programs.
4. Write programs: (a) leap year, (b) largest of three numbers, (c) grade from marks using else-if ladder.
5. Write programs: (a) factorial, (b) Fibonacci series, (c) prime check, (d) palindrome number.
6. Write a program to print the pyramid and the right-angled star triangle for n rows.
7. Explain break and continue with examples and a flowchart.
8. Trace the output: `for(i=1;i<=3;i++) for(j=1;j<=i;j++) printf("%d",j);`